Kotlin Avanzado

Colecciones

Kotlin proporciona su API de colecciones como una biblioteca estándar construida sobre la API de colecciones de Java como *'ArrayList'*, *'Maps'*, etc... Kotlin tiene dos variantes de colecciones: **mutables** e **inmutables**. Una colección mutable nos brinda la capacidad de modificar una colección ya sea agregando, eliminando o reemplazando un elemento. Las colecciones inmutables no se pueden modificar y no tienen estos métodos de ayuda.

Lists - [Inmutable]

Una lista es una colección ordenada de elementos. Esta es una colección popular ampliamente utilizada.

Podemos crear una **lista inmutable** usando la función `listOf()`. Los elementos no se pueden agregar ni eliminar.

```
val fooList = listOf("a", "b", "c", 1, false)
val numbers: List<Int> = listOf(1, 2, 3, 4)
val emptyList: List<String> = emptyList<String>() // lista vacía
val nonNullsList: List<String> = listOfNotNull(2, 45, 2, null, 5,
null) // lista de valores no nulos

println(fooList.size) // => 3
println(fooList.first()) // => a
println(fooList.last()) // => c
println(fooList.indexOf("b")) // 1

// Se puede acceder a los elementos de una lista por su índice
println(fooList[1]) // => b
```

Se puede crear una **lista mutable** utilizando la función `mutableListOf()`:

```
val fooMutableList = mutableListOf("a", "b", "c")
fooMutableList.add("d")
println(fooMutableList.last()) // => d
println(fooMutableList.size) // => 4
```

Con la función `'arrayListOf()'` crea una lista mutable y devuelve un tipo *'ArrayList'* de la API de colecciones de Java.

Sets - [Inmutable]

Un conjunto o 'set' es una colección desordenada de elementos únicos. En otras palabras, es una colección que no admite duplicados.

Podemos crear un conjunto (o 'set') inmutable utilizando la función `'setOf()'`:

```
val fooSet = setOf("a", "b", "c")
println(fooSet.contains("a")) // => true
println(fooSet.contains("z")) // => false
```

Con la función `'mutableSetOf()'` podemos crear un conjunto mutable:

```
// creates a mutable set of int types only
val intsMutableSet: MutableSet<Int> = mutableSetOf(3, 5, 6, 2, 0)
intsMutableSet.add(8)
intsMutableSet.remove(3)
```

La función `'hashSetOf()'` retorna un *'HashSet'* de la API de colecciones de Java el cual almacena los elementos en una tabla 'hash'. Podemos añadir o quitar elementos de este conjunto porque es **mutable**.

La función `'linkedSetOf()'` retorna un *'LinkedHashSet'* de la API de colecciones de Java. También es un conjunto mutable.

Maps - [Inmutable]

Los mapas asocian una clave a un valor. Las claves deben ser únicas, y por tanto no se permite duplicados. En cambio no hay obligación de que los valores asociados sean únicos. Cada clave sólo podrá asociarse a un solo elemento. De esa manera, cada clave se puede usar para identificar de forma única el valor asociado, ya que el mapa se asegura de que no pueda haber claves duplicadas en la colección. Los mapas implementan un forma eficiente de obtener el valor correspondiente a una determinada clave.

Podemos crear un **mapa ('map') inmutable** usando la función `'mapOf()'`:

```
val fooMap = mapOf("a" to 8, "b" to 7, "c" to 9)

// Se puede acceder a los valores en el mapa por su clave
println(fooMap["a"]) // => 8

// iterar por un mapa con un bucle 'for'
for ((key, value) in fooMap) {
    println("Key $key and value $value")
}
```

La función `'linkedHashMap()'` retorna un *'LinkedHasMap'* de la API de colecciones de Java, que es **mutable**.

La función `'sortedMapOf()'` retorna un *'SortedMap'* de la API de colecciones de Java que también es **mutable**.

[Sequences]

Las secuencias representan colecciones *'lazily-evaluated'*. Podemos crear una secuencia utilizando la función `'generateSequence()'`. Las secuencias son excelentes cuando el tamaño de la colección es desconocido a priori:

```
val fooSequence = generateSequence(1, { it + 1 })
val x = fooSequence.take(10).toList()
println(x) // => [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

// An example of using a sequence to generate Fibonacci numbers:
```

```
fun fibonacciSequence(): Sequence<Long> {
    var a = 0L
    var b = 1L
    fun next(): Long {
        val result = a + b
        a = b
        b = result
        return a
    }
    return generateSequence(::next)
}

val y = fibonacciSequence().take(10).toList()
println(y) // => [1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
```

Kotlin proporciona *'higher-order functions'* para trabajar con colecciones:

```
val z = (1..9).map { it * 3 }
        .filter { it < 20 }
        .groupBy { it % 2 == 0 }
        .mapKeys { if (it.key) "even" else "odd" }

println(z) // => {odd=[3, 9, 15], even=[6, 12, 18]}
```

Destructuring data

Los objetos pueden ser desestructurados en múltiples variables. Esta sintaxis se llama **declaración de desestructuración**. Una declaración de desestructuración crea múltiples variables a la vez.

```
val (a, b, c) = fooCopy
println("$a $b $c") // => 1 100 4
```

Desestructurando en un bucle `'for'`:

```
for ((a, b, c) in listOf(fooData)) {
    println("$a $b $c") // => 1 100 4
}

val mapData = mapOf("a" to 1, "b" to 2)
// Map.Entry is destructurable as well
for ((key, value) in mapData) {
    println("$key -> $value")
}
```

Rangos de valores

Un rango se define como un intervalo que tiene un valor de inicio y un valor final. **Los rangos son cerrados**, lo que significa que el valor inicial y final están incluidos en el rango. Los rangos se crean con el operados `..` o con funciones como `rangeTo()` o `downTo()`.

Para crear un intervalo sin incluir el último elemento usamos la función `until`.

```
val oneToNine = 1..9
val oneToFive: IntRange = 1.rangeTo(5)
```

```
val fiveToOne = 5.downTo(1)
print(fiveToOne)  // => 5 downTo 1 step 1

val oneToTen = (1..10).step(2).reversed() // => 9, 7, 5, 3, 1
println("${tenToOne.first} - ${tenToOne.last}") // => 10 - 1

val oneToFour = 1.until(5)
print(r) // => 1..4
```

Los tipos `IntRange`, `LongRange`, `CharRange` tienen una característica extra y es que permite iterar sobre los intervalos.

Una vez que se crea un intervalo, se puede usar el operador `in` para probar si un valor dado está incluido en el intervalo o el operador `!in` para comprobar si un valor no está en el intervalo:

```
// Iterar con un bucle 'for'
for (i in 1..10) { // equivalent of 1 <= i && i <= 10
    print(i)
}

// Iterar en sentido inverso
for (i in 4 downTo 1) {
    print(i)
}

// Iterar por un intervalo sin incluir el último elemento
for (i in 1 until 10) {
    // i in [1, 10), 10 is excluded
    println(i)
}

// Pasos arbitrarios
for (i in 1..4 step 2) {
    print(i)
}

for (i in 4 downTo 1 step 2) {
    print(i)
}
```

Smart Casting

Podemos verificar si un objeto es de un tipo en particular usando el operador `is` o si no es de un tipo con el operador `!is`.

Si un objeto pasa una verificación de tipo entonces se puede usar como ese tipo sin realizar la conversión explícitamente:

```
fun smartCastExample(x: Any): Boolean {
    if (x is Boolean) {
        // x is automatically cast to Boolean
        return x
    } else if (x is Int) {
        // x is automatically cast to Int
        return x > 0
```

```
    } else if (x is String) {
        // x is automatically cast to String
        return x.isNotEmpty()
    } else {
        return false
    }
}
println(smartCastExample("Hello, world!")) // => true
println(smartCastExample("")) // => false
println(smartCastExample(5)) // => true
println(smartCastExample(0)) // => false
println(smartCastExample(true)) // => true
```

La conversión inteligente (*'smart cast'*) también funciona con bloques `when` o bucles `while`:

```
fun smartCastWhenExample(x: Any) = when (x) {
    is Boolean -> x
    is Int -> x > 0
    is String -> x.isNotEmpty()
    else -> false
}
```

Conversión explícita o *'Explicit Casting'*

Podemos usar el operador `as` (o el operador de **conversión no segura** o *'unsafe cast operator'*) para convertir explícitamente una referencia de un tipo a otro tipo en Kotlin. Si la operación de conversión explícita es ilegal, tenga en cuenta que se lanzará una excepción de tipo *'ClassCastException'*.

Para evitar que se lance una excepción al realizar la conversión, podemos usar el operador de **conversión seguro** `as?`. Este operador intentará la conversión y si no se puede realizar la conversión devolverá `'null'` en vez de lanzar la excepción. Por tanto la variable que contiene el resultado de una conversión segura debe ser capaz de mantener un resultado nulo:

```
val circle = shape as Circle

val circle: Circle? = shape as? Circle // Conversión segura
```

Valores nulos ('nullable types')

Para que una variable contenga el valor '**null**' debe especificarse explícitamente como *'nullable'*. Una variable se puede especificar como *'nullable'* agregando un `?` a su tipo.

Podemos acceder a una variable o método *'nullable'* utilizando el operador `'?.'` también llamado *'Safe Call Operator'*. Un método o variable sólo será invocado si tiene una valor no nulo. En caso de que sea nulo será ignorado evitando un *'NullPointerException'*

Kotlin provee el operador `'?:'`, también llamado *'Elvis Operator'* para especificar un valor alternativo para usar si una variable es nula. Cuando la expresión de la izquierda del operador `'?:'` no es nulo entonces lo devuelve. En caso de que sea nulo devuelve la

expresión de la derecha. La expresión de la derecha sólo será evaluada si la expresión de la izquierda es 'null'.

```
val name: String = null // no compilará ya que no puede contener valores nulos
var fooNullable: String? = "abc"

fooNullable?.length // => 3

// 'Elvis Operator'
fooNullable?.length ?: -1 // => 3

fooNullable = null
val len: Int? = fooNullable?.length // El tipo de retorno de 'fooNullable' puede ser 'null' y por tanto debemos usar Int?

fooNullable?.length // => null
fooNullable?.length ?: -1 // => -1

// Encadenar 'safe calls'. La cadena retorna 'null' si alguna de ellas es 'null'
fun getCountryNameSafe(person: Person?): String? {
    return person?.address?.city?.country?.name
}

// Dado que 'throw' y 'return' son expresiones en Kotlin se pueden usar en la parte derecha del operador 'Elvis'
fun foo(node: Node): String? {
    val parent = node.getParent() ?: return null
    val name = node.getName() ?: throw IllegalArgumentException("name expected")
    // ...
}
```

De manera similar, podemos devolver tipos *'nullable'* y no *'nullable'* desde una función.

```
fun getName(): String? = name // Esta función puede o no devolver una referencia nula.

fun getNotNullName(): String = name ?: "John" // Esta función no devolverá una referencia nula

getName() // => null
getNotNullName() // => John
```

Con *'smart cast'*, el compilador rastrea las condiciones dentro de una expresión `'if'`. Si realizamos la verificación de que una variable no es nula, entonces el compilador nos permitirá acceder a la variable como si hubiera sido declarada como un tipo no anulable:

```
var l = if (name != null) name.length else -1
```

El operador de aserción no-nulo `'!!'` convierte cualquier valor a un tipo no nulo y lanza una excepción *'NullPointerException'* si el valor es nulo.

```
val length: Int = name!!.length
```

Igualdad

En Kotlin hay tenemos la **igualdad estructural** y la **igualdad referencial**.

La igualdad estructural se comprueba con la operación `'=='` y la parte contraria `'!='` y se utiliza para comprobar si dos valores o variables son iguales (`equals()`)

```
if (a == b) {
    // ...
} else {
    // ...
}
```

La igualdad referencial se comprueba con la operación `'===='` y su contraparte `'!=='` y evalúa a `true` si y sólo si dos referencias apuntan al mismo objeto.

[Standard Library Functions]

Son funciones que proporciona Kotlin para aumentar la biblioteca estándar de Java.

[Apply]

`'apply'` es una función de extensión de la biblioteca estándar de Kotlin declarada en `'Any'`, por lo que puede ser invocada en cualquier tipo de instancia. `'apply'` acepta una expresión lambda que es invocada y el receptor es la instancia donde es llamada. La función `'apply'` devuelve una instacia del original.

Su uso principal es hacer que el código que necesita inicializar una instancia sea más legible permitiendo que las funciones y las propiedades se llamen directamente dentro de la función antes de devolver el valor en sí.

```
data class Person(var firstName: String, var lastName : String)
var person = Person("John", "Doe")

person.apply { this.firstName = "Bruce" }
print(person) // => Person(firstName=Bruce, lastName=Doe)

// 'apply' retorna la instancia original.
person.apply { this.firstName = "Bruce" }.firstName = "Steve"
print(person) // => Person(firstName=Steve, lastName=Doe)
```

[Let]

La función `'let'` toma el objeto sobre el que se invoca como parámetro y devuelve el resultado de la expresión lambda. Es útil cuando desea ejecutar algún código en un objeto antes de devolver algún valor diferente y no necesita mantener una referencia al original:

```
fun main(args: Array<String>) {
    var str = "Hello World"
    str.let { println("$it!!") } // => Hello World!!
    println(str) // => Hello World
}

var strLength = str.let { "$it function".length } // devuelve el
resultado de la expresión lambda
```

```
println("strLength is $strLength") // => strLength is 25
```

[With]

La función `'with'` es una función de nivel superior diseñada para los casos en los que desea llamar a múltiples funciones en un objeto y no desea repetir el receptor cada vez. La función `'with'` acepta un receptor y un cierre para operar en dicho receptor:

```
data class Person(var firstName: String, var lastName : String)
var person = Person("John", "Doe")

with(person)
{
    firstName = "Bruce"
    lastName = "Doe"
}

// notación sin 'with'
person.firstName = "John"
person.lastName = "Doe"
```

La última expresión en un bloque `'with'` se retorna como resultado:

```
var name = with(person)
{
    firstName = "John"
    lastName = "Doe"
    "$firstName $lastName" // se retorna este valor y se almacena en
'name'
}
println(name) // => John Doe
```

[Run]

`'Run'` es una función que combina las características de `'with'` y `'let'`. Esto significa que se pasa una expresión lambda a la función `'run'` y la instancia del objeto es el receptor. El valor de retorno de la expresión lambda se usa como valor de retorno:

```
person.run {
    this.firstName = "Bruce"
}
print(person) // => Person(firstName=Bruce, lastName=Doe)
```

La diferencia clave entre `'let'` y `'run'` es que con `'run'` el receptor es la instancia, mientras que en `'let'`, el argumento de la expresión lambda es la instancia.

[Repeat]

Esta función acepta un entero y una función literal. La función literal será invocada las veces indicadas por el valor entero.

```
repeat(10, { println("Hello") })
```

[Lazy]

La función `'lazy'` es una función cuya utilidad es envolver funciones costosas en términos de rendimiento o de recursos y que serán invocadas cuando sean requeridas por primera vez. La ventaja de utilizar esta función proporcionada por la biblioteca estándar de Kotlin es que el compilador mantendrá la invocación sincronizada evitando que sea invocada más de una vez.

```
fun readStringFromDatabase(): String = ... // expensive operation
val lazyString = lazy { readStringFromDatabase() }
```

[Use]

La función `'use'` es similar a la declaración `'try-with-resources'` presente en Java 7. La función `'use'` se define como una función de extensión de la interfaz *'Closeable'*. Ejecuta la función y luego 'cierra' el recurso de forma segura.

Assertions

Kotlin proporciona un conjunto de funciones que nos permiten agregar una cantidad limitada de **especificaciones formales** a nuestro código. Una especificación formal es una aserción que siempre debe ser verdadera o falsa en la ubicación cuando se ejecuta la aserción. Estos también se conocen como contratos o diseño por contrato:

- `'require()'` y `'requireNotNull()'` lanza una excepción de tipo *'IllegalArgumentException'* y se utiliza para garantizar que los argumentos cumplan el contrato.
- `'assert()'` lanza una excepción *'AssertionException'* y se utiliza para garantizar que nuestro estado interno es consistente.
- `'check()'` y `'error()'` lanza una excepción *'IllegalStateException'* y también se usa para mantener la consistencia del estado interno.

Estas funciones son similares. La clave que las diferencia es el tipo de excepción que se plantea.

```
fun neverEmpty(str: String) {
    require(str.length > 0, { "String should not be empty" })
    println(str)
}

fun foo(k: Int, value: Boolean) {
    require(k > 10, { "k should be greater than 10" }) // => throws an
IllegalArgumentException
    requireNotNull(k) // => throws an IllegalArgumentException if the
value is null.
    check(value) // => throws an IllegalStateException if the value is
false
    if (k == 20) error("Error: k == 20") // => throws an
IllegalStateException
}
```

Excepciones

En Kotlin todas las excepciones son subclases de la clase `'Throwable'`. Cada excepción tiene un mensaje, un seguimiento de la pila y una causa opcional. **Kotlin no**

tiene *'checked exceptions'* a diferencia de Java, que realiza la distinción entre tipos de excepciones.

Para lanzar un objeto de excepción, se utiliza la palabra clave `'throw'`:

```
throw Exception("Message")
```

Para capturar una excepción lanzada se utiliza un bloque `'try'`:

```
try {
    // some code
}
catch (e: SomeException) {
    // handler
}
finally {
    // optional finally block
}
```

Puede haber 0 o más bloques `'catch'`. Los bloques `'finally'` son opcionales y puede omitirse. Sin embargo, tiene que haber al menos un bloque `'catch'` o `'finally'`.

Al igual que muchas otras instrucciones en Kotlin, `'try'` es una expresión y por tanto puede devolver un valor:

```
val a: Int? = try { parseInt(input) } catch (e: NumberFormatException)
{ null }
```

El valor devuelto por un `'try'` que actúa como expresión es la última expresión en el bloque `'try'` o la última expresión en el bloque `'catch'`. El contenido del bloque `'finally'` no afecta al resultado de la expresión.

`'throw'` es una expresión en Kotlin, así que se puede usar, por ejemplo, como parte de una *'Elvis expression'*:

```
val s = person.name ?: throw IllegalArgumentException("Name required")
```

El tipo de retorno de una expresión `'throw'` es el tipo especial `'Nothing'`. Este tipo no tiene valores y se utiliza para marcar ubicaciones del código que nunca se pueden alcanzar.

```
fun fail(message: String): Nothing {
    throw IllegalArgumentException(message)
}
```

Cuando llame a la función del ejemplo anterior, el compilador sabrá que la ejecución no continúa más allá de la llamada:

```
val s = person.name ?: fail("Name required")
println(s)     // 's' is known to be initialized at this point
```

Otro caso en el que puede encontrar este tipo es la inferencia de tipos. La variante *'nullable'* de este tipo, `'Nothing?'`, tiene exactamente un valor posible, que es el valor

'null'. Si se usa el valor nulo para inicializar un valor de un tipo inferido y no hay otra información que se pueda usar para determinar un tipo más específico, el compilador inferirá el tipo 'Nothing?':

```
val x = null           // 'x' tiene el tipo `Nothing?`
val l = listOf(null)   // 'l' tiene el tipo `List<Nothing?>
```

Anotaciones

Las **anotaciones** permiten a los desarrolladores agregar un significado adicional a las clases, interfaces, parámetros, etc., en el momento de la compilación. Las anotaciones pueden ser utilizadas por el compilador o por su propio código a través de la reflexión en tiempo de ejecución. Dependiendo del valor de la anotación, el significado del programa o los datos puede cambiar.

[@JvmStatic]

Kotlin representa funciones de nivel de paquete (funciones fuera de una clase) como métodos estáticos. Kotlin también puede generar métodos estáticos para funciones definidas en ***'objects'*** y ***'companin objects'*** si anota esas funciones como '@JvmStatic'. Si usa esta anotación, el compilador generará tanto un método estático en la clase envolvente del objeto como un método de instancia en el propio objeto.

```
class C {
    companion object {
        @JvmStatic fun foo() {}
        fun bar() {}
    }
}

// Ahora 'foo()' es estático en Java pero no 'bar()'
C.foo(); // correcto
C.bar(); // error: 'bar()' no es un método estático
C.Companion.foo(); // correcto
C.Companion.bar(); // la única forma de invocar a 'bar()'

object Obj {
    @JvmStatic fun foo() {}
    fun bar() {}
}

// In Java:
Obj.foo(); // correcto
Obj.bar(); // error
Obj.INSTANCE.bar(); // correcto, una llamada a través de la instancia
'Singleton'
Obj.INSTANCE.foo(); // correcto
```

[@Throws]

Dado que todas las excepciones en Kotlin son *'unchecked exceptions'*, no es necesario agregar una lista de posibles excepciones a las firmas de métodos como las que hay en Java. Sin embargo, es posible que deseamos informar a los usuarios de Java que nuestra API produce excepciones en ciertas situaciones. Podemos hacer esto utilizando la

anotación `'@Throws'`, que se utiliza para indicar al compilador que genere cláusulas de lanzamiento en los métodos generados.

```
@Throws(FileNotFoundException::class)
fun fileExists(path: String) {
    // ...
}
```

[@JvmOverloads]

Dada una función con parámetros por defecto, `'@JvmOverloads'` hará que el compilador cree múltiples métodos sobrecargados para cada parámetro predeterminado.

[@JvmName]

Podemos cambiar el nombre del fichero creado por Kotlin con la anotación `'@JvmName'`:

```
// example.kt (sin @JvmName)
package demo

class Foo

fun bar() { ... }

// En Java
new demo.Foo();
demo.ExampleKt.bar();


// Usamos la anotación '@JvmName' al principio del fichero para indicar al compilador el nombre del fichero
@file:JvmName("DemoUtils")

package demo

class Foo

fun bar() { ... }

// Ahora en Java
new demo.Foo();
demo.DemoUtils.bar();
```

[@JvmMultifileClass]

Además de indicarle al compilador el nombre del fichero con `'@JvmName'` podemos indicarle que combine todas las funciones de nivel superior de varios ficheros en Kotlin en una única clase Java con la anotación `'@JvmMultifileClass'`.

[Reflection]

Reflection es el nombre dado a la inspección del código en tiempo de ejecución en lugar de tiempo de compilación. Puede usarse para crear instancias de clases, buscar

funciones e invocarlas, inspeccionar anotaciones, buscar campos y descubrir parámetros y genéricos, todo sin conocer esos detalles en el momento de la compilación.

Por ejemplo, si necesitamos persistir tipos en una base de datos y a priori no conocemos el tipo de datos podemos utilizar la reflexión para conocer el tipo de datos en tiempo de ejecución y crear la SQL apropiada a ese tipo.

Para usar la reflexión en Kotlin hay que importar el paquete `kotlin.reflect`.

`'KClass'` es el tipo central utilizado en la reflexión de Kotlin. Cada tipo tiene una instancia de `'KClass'` en tiempo de ejecución que contiene detalles de las funciones, propiedades, anotaciones, etc., para ese tipo. Para obtener una instancia de `'KClass'` para cualquier tipo, usamos la sintaxis especial `'::class'` en una instancia de ese tipo:

```
val name = "George"
val kclass = name::class // => class kotlin.String

data class Person(val firstName: String, val lastName: String)
println(Person::class.qualifiedName) // => Person
println(Person::class.isData) // => true
```

Podemos obtener una referencia a la clase utilizando el *'fully qualified name or FQN'* de la clase y la API 'reflection' de Java. Si el compilador no encuentra la clase lanza una *'ClassNotFoundException'*:

```
package com.example
data class Person(val firstName: String, val lastName: String)

val kClass = Class.forName("com.example.Person").kotlin // => class com.example.Personal
```

Para crear instancias de tipo sin conocer el tipo en tiempo de ejecución podemos invocar la función `'createInstance()'` en una referencia de `'KClass'`. Podemos usar esta función con clases sin parámetros o con parámetros opcionales, es decir, que tengan valor por defecto:

```
class PositiveInteger(value: Int = 0)

fun createInteger(kclass: KClass<PositiveInteger>): PositiveInteger {
    return kclass.createInstance()
}
```

Podemos devolver una lista de todos los constructores declarados en un tipo dado usando la propiedad `'constructor'` disponible en el tipo `'KClass'`. Podemos instanciar una clase usando el constructor con la instrucción `'call'` o `'callBy'`:

```
class Person constructor(val firstName: String, val lastName: String)

fun <T : Any> printConstructors(kclass: KClass<T>) {
    kclass.constructors.forEach {
        println(it.parameters)
    }
}
```

```
printConstructors(Person::class) // Muestra el/los constructor/es de la clase 'Person'

// Recupera el primer constructor. Si no encuentra ninguno lanza una excepción.
val constructor = Person::class.constructors.first()
val person = constructor.call("John", "Doe") // Invocar al constructor con 'call'
println(person.firstName) // => John
```

Además de los constructores de una clase, también podemos acceder y listar las funciones de una clase con la propiedad `'functions'` disponible en el tipo `'KClass'`:

```
class Person constructor(val firstName: String, val lastName: String) {
    fun getName(): String {
        return "$firstName $lastName"
    }
}

fun <T : Any> printFunctions(kclass: KClass<T>) {
    kclass.functions.forEach {
        println(it.name)
    }
}

printFunctions(Person::class) // => getName equals hashCode toString

val function = Person::class.functions.find { it.name == "getName" }
val person = Person("John", "Doe")
function?.call(person) // => John Doe
```

[Coroutines]

Testing

KotlinTest es el framework para probar y testear el código en Kotlin. Añadir la dependencia a Gradle: `testCompile 'io.kotlintest:kotlintest:x.y.z'`.

Normalmente, para mantener ordenada la estructura del proyecto los ficheros de test se ubican en `src/test/kotlin`

Una especificación o *'spec'* es simplemente la manera en que las pruebas se presentan en los archivos de clase. Hay varias especificaciones diferentes disponibles como **FunSpec**, **StringSpec+*, **ShouldSpec**. etc...

La especificación **FunSpec** permite crear pruebas similares al estilo *jUnit*. Para escribir un test unitario invocamos la función *'test'* que toma dos parámetros. El primer parámetro es una descripción de la prueba unitaria y el segundo es una función literal que contiene el cuerpo de la prueba. La descripción o nombre de la prueba aparecerá en la salida, así que permite saber que prueba/s han pasado la prueba y cuáles han fallado.

```
class StringTestWithFunSpec : FunSpec() {
    init {
```

```
        test("String.startsWith should be true for a prefix") {
            "helloworld".startsWith("hello") shouldBe true
        }
        test("String.endsWith should be true for a prefix") {
            "helloworld".endsWith("world") shouldBe true
        }

    }
}
```

La especificación **StringSpec** es la especificación recomendada por los autores de Kotlin y es la especificación más simple y compacta ya que reduce la sintaxis al mínimo. Se escribe una cadena seguida de una expresión lambda para probar el código:

```
class StringTestWithStringSpec : StringSpec() {
    init {
        "strings.length should return size of string" {
            "hello".length shouldBe 5
            "hello" shouldBe haveLength(5)
        }
    }
}
```

La especificación **ShouldSpec** es similar a **FunSpec** pero usa la palabra clave `'should'` en vez de `'test'`:

```
class StringTestWithShouldSpec : ShouldSpec() {
    init {
        should("return the length of the string") {
            "sammy".length shouldBe 5
            "".length shouldBe 0
        }
        // Nested form
        "String.length" {
            should("return the length of the string") {
                "sammy".length shouldBe 5
                "".length shouldBe 0
            }
        }
    }
}
```

La especificación **WordSpec** usa también la palabra clave `'should'`. Esta especificación permite anidar las pruebas:

```
class StringTestWithWordSpec : WordSpec() {
    init {
        "String.length" should {
            "return the length of the string" {
                "sammy".length shouldBe 5
                "".length shouldBe 0
            }
        }
    }
}
```

La especificación **BehaviorSpec** utiliza las palabras clave `'given'`, `'when'` y `'then'` para crear pruebas unitarias más cercanas al lenguaje natural:

```
class StringTestWithBehaviorSpec : BehaviorSpec() {
    init {
        given("a stack") {
            val stack = Stack<String>()
            `when`("an item is pushed") {
                stack.push("kotlin")
                then("the stack should not be empty") {
                    stack.isEmpty() shouldBe true
                }
            }
            `when`("the stack is popped") {
                stack.pop()
                then("it should be empty") {
                    stack.isEmpty() shouldBe false
                }
            }
        }
    }
}
```

La especificación **FeatureSpec** es similar a la especificación **BehaviorSpec** pero utiliza las palabras clave `'feature'` y `'scenario'`:

```
class StringTestWithFeatureSpec : FeatureSpec() {
    init {
        feature("Hello World") {
            scenario("should starts with 'Hello'") {
                "Hello World".startsWith("Hello")
            }
            scenario("should ends with 'World'") {
                "Hello World".endsWith("World")
            }
        }
    }
}
```

Los **matchers** prueban alguna propiedad, indicada por el nombre del **matcher**, más allá de la simple igualdad. Por ejemplo, un comparador puede verificar si una cadena está vacía o si un entero es positivo.

```
// [String matchers]
class StringTestWithDifferentMatchers : StringSpec() {
    init {
        "Tests string prefixes" {
            "Hello".startsWith("He") shouldBe true
            "Hello" shouldBe startWith("He")
        }
        "Tests substrings"{
            "Hello" shouldBe include("el")
        }
        "Test string suffixes" {
            "Hello".endsWith("llo") shouldBe true
            "Hello" shouldBe endWith("llo")
        }
        "Tests the length of a string" {
            "Hello".length shouldBe 5
            "Hello" shouldBe haveLength(5)
        }
        "Tests the equality using a regular expression" {
```

```
            "Hello" shouldBe match("He...")
        }
    }
}

// [Collection matchers]
class CollectionTestWithDifferentMatchers : StringSpec() {
    private val listWithDifferentIntegers = listOf(1, 2, 3, 4, 5)
    private val mapWithKeyAndValues = mapOf<Int, String>(1 to "Hello",
2 to "World")

    init {
        "Tests that a collection should contain the given element" {
            listWithDifferentIntegers shouldBe contain(3)
        }
        "Test the size of the collection" {
            listWithDifferentIntegers shouldBe haveSize<Int>(5)
        }
        "Tests that the collections should be sorted" {
            listWithDifferentIntegers shouldBe sorted<Int>()
        }
        "Tests that the collection has a single element that is equal
to the given element" {
            listWithDifferentIntegers shouldNotBe singleElement(2)
        }
        "Tests that the collection contains all the given elements.
The order of these elements does not matter." {
            listWithDifferentIntegers shouldBe containsAll(1, 2, 4)
        }
        "Tests whether the collection is empty or not" {
            listWithDifferentIntegers shouldNotBe beEmpty<Int>()
        }
        "Tests whether the map contains mapping from a key to any
value" {
            mapWithKeyAndValues shouldBe haveKey(2)
        }
        "Tests whether the map contains the value for at least one
key" {
            mapWithKeyAndValues shouldBe haveValue("Hello")
        }
        "Tests that the map contains the exact mapping of the key to
the value" {
            mapWithKeyAndValues shouldBe contain(2, "World")
        }
    }
}

// [Floating point matchers]
// En valores en punto flotante más que la igualdad absoluta se
utiliza la 'tolerancia' que es el valor mínimo entre dos valores que
satisfacen el criterio de igualdad
class FloatNumberTestWithTolerance : StringSpec() {
    private val randomDouble = 18.005
    private val enoughDouble = 18.006

    init {
        "Test if two numbers are equals" {
            randomDouble shouldNotBe equals(enoughDouble)
            randomDouble shouldBe (enoughDouble plusOrMinus 0.01)
        }
    }
```

```
}

// [Exception matchers]
// 'shouldThrow fallará si se lanza una excepción diferente
class ExceptionTest : StringSpec() {
    init {
        "Testing IllegalArgumentException" {
            shouldThrow<IllegalArgumentException> {
                addNumberToTwo(10.0) shouldEqual 10.5
            }
        }
    }
}

@Throws(IllegalArgumentException::class)
fun addNumberToTwo(a: Any): Int {
    if (a !is Int) {
        throw IllegalArgumentException("Number must be an integer")
    }
    return 2 + a
}
```

Los **matchers** se pueden combinar usando los operadores de la lógica booleana como `'and'` y `'or'`:

```
class CombiningMatchers : StringSpec() {
    init {
        "Combining matchers" {
            "Hello World" should (startWith("Hel") and endWith("rld"))
        }
    }
}
```

Un **inspector** en KotlinTest es la forma más fácil de probar el contenido de *'collections'*:

```
val kings = listOf("Stephen I", "Henry I", "Henry II", "Henry III",
"William I", "William III")

class InspectorTests : StringSpec() {
    init {
        "all kings should have a regal number" {
            forAll(kings) {
                it should endWith("I")
            }
        }
        "only one king has the name Stephen" {
            forOne(kings) {
                it should startWith("Stephen")
            }
        }
        "some kings have regal number II" {
            forSome(kings) {
                it should endWith("II")
            }
        }
        "at least one King has the name Henry" {
            forAtLeastOne(kings) {
                it should startWith("Henry")
            }
        }
```

```
    }
}
```

A veces es posible que sea necesario ejecutar algo de código, antes de que se ejecuten las pruebas o después de que se completen todas las pruebas (sean exitosas o no). Esto se puede lograr mediante el uso de la clase abstracta `'ProjectConfig'`. Para usar esto, simplemente se crea un objeto que extienda de esta clase abstracta y asegurarse que esté en la ruta de la clase. KotlinTest lo encontrará automáticamente y lo invocará:

```
object codeExecutionBeforeAndAfterTestCases : ProjectConfig() {
    override fun beforeAll() {
        // ...code
    }

    override fun afterAll() {
        // ...code
    }
}
```

Java Interop

Calling Java from Kotlin

Kotlin está diseñado teniendo en cuenta la interoperabilidad de Java. El código Java existente puede llamarse desde Kotlin de una manera natural, y el código Kotlin también se puede usar desde Java sin problemas.

Casi todo el código de Java se puede utilizar sin problemas:

```
import java.util.*

fun demo(source: List<Int>) {
    val list = ArrayList<Int>()
    // 'for'-loops work for Java collections:
    for (item in source) {
        list.add(item)
    }
    // Operator conventions work as well:
    for (i in 0..source.size - 1) {
        list[i] = source[i] // get and set are called
    }
}
```

'Getters' and 'Setters'

Los métodos que siguen las convenciones de Java para *'getters'* y *'setters'* (métodos sin argumentos con nombres que comienzan con 'get' y métodos con argumentos únicos con nombres que comienzan con 'set') se representan como **propiedades** en Kotlin.

Los métodos de acceso booleanos (donde el nombre del *'getter'* comienza con 'is' y el nombre del *'setter'* comienza con 'set') se representan como propiedades que tienen el mismo nombre que el método *'getter'*:

```
import java.util.Calendar
```

```
fun calendarDemo() {
    val calendar = Calendar.getInstance()
    if  (calendar.firstDayOfWeek  ==  Calendar.SUNDAY)  {     //  call
getFirstDayOfWeek()
        calendar.firstDayOfWeek  =  Calendar.MONDAY            //  call
setFirstDayOfWeek()
    }
    if  (!calendar.isLenient)  {                                 //  call
isLenient()
        calendar.isLenient  =  true                              //  call
setLenient()
    }
}
```

Si la clase Java solo tiene un *'setter'*, no será visible como una propiedad en Kotlin, ya que Kotlin no admite propiedades que tengan únicamente el método *'setter'*.

'Void' como retorno

Si un método Java devuelve `'void'`, devolverá `'Unit'` cuando se llame desde Kotlin. Si, por casualidad, alguien usa ese valor de retorno, el compilador de Kotlin lo asignará en el sitio de la llamada, ya que el valor en sí mismo se conoce de antemano (es `'Unit'`).

Escapar palabras clave en Kotlin

Algunas de las palabras clave de Kotlin son identificadores válidos en Java, como por ejemplo `'in'`, `'object'`, `'is'`, etc... Si una biblioteca de Java usa una palabra clave de Kotlin para un método, se puede escapar usando las comillas invertidas (`):

```
// Java
public class Date {
    public void when(str:String) { .... }
}

// Kotlin
date.`when`("2016")
```

Null-Safety

Cualquier referencia en Java puede ser nula, lo que hace que los requisitos de Kotlin de seguridad con los valores nulos no sean prácticos para los objetos procedentes de Java. Los tipos de declaraciones de Java se tratan especialmente en Kotlin y se llaman `'platform types'`. Los controles nulos son relajados para tales tipos, por lo que las garantías de seguridad para ellos son las mismas que en Java.

```
val list = ArrayList<String>() // non-null (constructor result)
list.add("Item")
val size = list.size // non-null (primitive int)
val item = list[0] // platform type inferred (ordinary Java object)

item.substring(1) // allowed, may throw an exception if item == null
```

'Checked exceptions'

Kotlin no tiene *'checked exceptions'*. Por lo tanto, los métodos Java que tienen *'checked exceptions'* se tratan de la misma manera que el resto de métodos.

Calling Kotlin from Java

Al igual que Java se puede usar sin problemas en Kotlin, Kotlin se puede usar fácilmente desde Java.

Top-level functions

La JVM no admite funciones de nivel superior. Por lo tanto, para hacer que funcionen con Java, el compilador Kotlin crea una clase Java con el nombre del paquete. Las funciones se definen luego como métodos estáticos Java en esta clase, que deben ser instanciados antes de su uso.

```
// Kotlin
package org.example.utils
fun cube(n: Int): Int = n * n * n

// Java
import org.example.utils.Utils;
UtilsKt.cube(3);
```

Como se indica en la sección de "Anotaciones", podemos indicar al compilador el nombre del fichero con la anotación `'@JvmName'`:

```
// Kotlin
@file:JvmName("Utils")
package org.example.utils
fun cube(n: Int): Int = n * n * n

// Java
import org.example.utils.Utils;
Utils.cube(3);
```

Default parameters

la JVM no tiene soporte para los parámetros por defecto. Por lo tanto, cuando una función se define con los valores predeterminados, el compilador debe crear una sola función sin los parámetros predeterminados. Sin embargo, podemos indicarle al compilador que cree múltiples sobrecargas de la función para cada parámetro predeterminado con la anotación `'@JvmOverloads'`. Luego, los usuarios de Java pueden ver las diversas funciones y elegir cuál es la más adecuada. Esta anotación funciona tanto para constructores, funciones o métodos estáticos:

```
// Kotlin
class Foo @JvmOverloads constructor(x: Int, y: Double = 0.0) {
    @JvmOverloads fun f(a: String, b: Int = 0, c: String = "abc") {
... }
}

// Java
```

```
// Constructors:
Foo(int x, double y)
Foo(int x)

// Methods
void f(String a, int b, String c) { }
void f(String a, int b) { }
void f(String a) { }
```

Objects and static methods

Los *'named objects'* y los *'companion objects'* se generan como instancias **'singleton'** de una clase. Sin embargo, podemos indicar al compilador que genere la función como una método estático en Java con la anotación `'@JvmStatic'`:

```
// Kotlin
object Console {
    fun clear() : Unit { } // Normal
    @JvmStatic fun exit() : Unit { } // Con anotación
}

// Java
Console.INSTANCE.clear() // Normal
Console.exit() // Con anotación
```

Checked exceptions

En Java, solo podemos detectar las *'checked exceptions'* si están declaradas en el método, incluso si el cuerpo del método lanza esa excepción. Por lo tanto, si tenemos una función que se utilizará desde Java y queremos permitir que las personas detecten una excepción, debemos informar al compilador para que agregue la excepción a la firma del método. Para ello usamos la anotación `'@Throws'`:

```
// Kotlin
@Throws(IOException::class)
fun createDirectory(file: File) {
    if (file.exists()) throw IOException("Directory already exists")
    file.createNewFile()
}

// Java
try {
    UtilsKt.createDirectory(new File("file.txt"));
} catch (IOException e) {
    // handle exception here
}
```

Reference

- https://kotlinlang.org/docs/reference/
- https://code.tutsplus.com/series/kotlin-from-scratch--cms-1209
- https://www.packtpub.com/application-development/programming-kotlin
- https://learnxinyminutes.com/docs/kotlin/
- https://gist.github.com/dodyg/5823184

- https://gist.github.com/dodyg/5616605
- https://github.com/Zhuinden/guide-to-kotlin
- https://superkotlin.com/kotlin-mega-tutorial/

Aclaraciones sobre [Kotlin]

En este punto vamos a ver algunas aclaraciones sobre el lenguaje de programación [Kotlin]. Se basa en las principales dudas que nos pueden surgir al empezar a programar en este lenguaje y que han sido extraídas de las preguntas que el alumnado ha realizado durante los distintos cursos.

1. Aclaraciones sobre el lenguaje

1.1. Funciones lambda

Imagina que tienes una lista de números y quieres quedarte solo con los que son pares. En Kotlin, puedes hacer esto fácilmente con `filter`.

La firma o signatura de la función `filter` en Kotlin es la siguiente:

```
fun <T> Iterable<T>.filter(predicate: (T) -> Boolean): List<T>
```

Aquí te desgloso lo que significa cada parte:

- `fun`: Indica que se trata de una función.
- `<T>`: Es un parámetro de tipo genérico. Esto significa que `filter` puede trabajar con listas de cualquier tipo de elementos, como `Int`, `String`, `YourCustomClass`, etc.
- `Iterable<T>`: Indica que `filter` es una función de extensión para la interfaz `Iterable<T>`. Dado que `List<T>` implementa `Iterable<T>`, puedes usar `filter` con cualquier lista.
- `predicate: (T) -> Boolean`: Es el parámetro que toma `filter`. Es una función lambda que toma un elemento de tipo `T` y devuelve un `Boolean`. Si la lambda devuelve `true`, el elemento se incluye en la lista resultante; si devuelve `false`, se excluye.
- `List<T>`: Es el tipo de retorno de `filter`. Devuelve una nueva lista que contiene solo los elementos que cumplen con el predicado.

Esta signatura refleja la versatilidad y el poder de `filter` en Kotlin: puedes usarla con cualquier tipo de lista y definir cualquier condición para filtrar los elementos de esa lista.

Por tanto, la función `filter` se utiliza para filtrar elementos de una lista basándose en una condición que defines. Funciona así: pasas una lambda (un tipo de función anónima) a `filter`, y esta lambda debe devolver `true` o `false` para cada elemento. Si devuelve `true`, el elemento se mantiene en la lista resultante; si devuelve `false`, se descarta.

Aquí tienes un ejemplo sencillo:

```
val numeros = listOf(1, 2, 3, 4, 5)
val pares = numeros.filter { it % 2 == 0 }
println(pares)  // Imprimirá: [2, 4]
```

En este código, `{ it % 2 == 0 }` es una lambda que chequea si un número es par. `it` es una referencia al elemento actual de la lista que se está procesando.

Ahora, sobre cómo está implementada `filter` internamente: es una función de extensión de la interfaz `List`. Utiliza un bucle para iterar sobre los elementos de la lista y aplica la lambda a cada uno. Si la lambda devuelve `true`, el elemento se agrega a una nueva lista. Finalmente, esta nueva lista filtrada se devuelve.

Si bien el código exacto de la implementación puede variar, conceptualmente es algo así:

```
fun <T> List<T>.filter(predicate: (T) -> Boolean): List<T> {
    val resultado = mutableListOf<T>()
    for (item in this) {
        if (predicate(item)) {
            resultado.add(item)
        }
    }
    return resultado
}
```

Espero que esto te ayude a entender mejor cómo funciona `filter` y en general cualquier función o método que utiliza lambdas entre sus parámetros.

1.2. Notación para llamar a las funciones cuando se utilizan lambdas

En Kotlin, cuando llamas a una función que toma una lambda como parámetro, puedes usar dos formas: con o sin paréntesis. La que ya mostré es la forma sin paréntesis, pero si prefieres usar paréntesis, puedes hacerlo también. Aquí te muestro cómo sería:

```
val numeros = listOf(1, 2, 3, 4, 5)
val pares = numeros.filter({ it % 2 == 0 }) //tambien se puede poner
numeros.filter { it % 2 == 0 }
println(pares)  // Esto imprimirá: [2, 4]
```

En esta versión, simplemente colocas la lambda entre paréntesis después de `filter`. Funciona exactamente igual que la versión sin paréntesis. Es más una cuestión de estilo y preferencia personal que de funcionalidad.

En Kotlin, cuando una función tiene una lambda como último parámetro, puedes optar por usar la "sintaxis de lambda fuera de los paréntesis", que es lo que hicimos en los ejemplos de más arriba. Pero si te sientes más cómodo con los paréntesis, ¡adelante!

1.3. ¿Qué es un receiver en kotlin?

En Kotlin, un "receiver" se refiere a un tipo especial de parámetro que está disponible dentro del cuerpo de una función de extensión o una lambda con receptor. Es una forma poderosa y flexible de añadir funcionalidades a clases existentes o de crear DSLs (Domain-Specific Languages). Veamos cada caso:

1. **Funciones de Extensión**: Cuando defines una función de extensión para una clase, el "receiver" es la instancia de esa clase sobre la cual se invoca la función. Por ejemplo:

```
fun String.exclamar() = this + "!"
```

   Aquí, `String` es el receiver. Dentro de exclamar, puedes usar this para referirte a la instancia específica de String sobre la que se llama a la función. Si haces `val saludo = "Hola".exclamar()`, `this` dentro de exclamar será `"Hola"`.

3. **Lambdas con Receptor**: Son similares a las funciones de extensión, pero en forma de lambda. Dentro de estas lambdas, tienes acceso directo a los métodos y propiedades del receiver. Un ejemplo común es con las builders en Kotlin, como en apply:

```
val lista = ArrayList<String>().apply {
    add("Uno")
    add("Dos")
}
```

   En este caso, `ArrayList<String>()` es el receiver para la lambda pasada a `apply`. Dentro de la lambda, puedes llamar directamente a métodos como `add` sin necesidad de especificar el objeto (no necesitas decir `this.add("Uno")`, simplemente `add("Uno")` funciona).

Por tanto, el receiver en Kotlin proporciona un contexto adicional dentro de una función de extensión o una lambda, permitiéndote acceder y modificar las propiedades y métodos del objeto sobre el cual se invoca la función o lambda. Esto hace que el código sea más conciso y legible, y es una parte fundamental de cómo Kotlin maneja muchas de sus características más elegantes y potentes.

1.4. ¿Qué es `it` en kotlin?

En Kotlin, `it` es un nombre implícito que se usa para referirse al parámetro único de una lambda cuando dicho parámetro no se declara de manera explícita. Es una forma conveniente y concisa de trabajar con lambdas que solo requieren un parámetro. Aquí te explico un poco más:

1. **Uso en Lambdas**: Cuando tienes una lambda que acepta un solo parámetro, puedes usar `it` para referirte a ese parámetro sin necesidad de declararlo. Por ejemplo: `kotlin val lista = listOf(1, 2, 3) val cuadrados = lista.map { it * it }`

En este caso, la lambda pasada a `map` tiene un parámetro (cada elemento de la lista), y usamos `it` para referirnos a ese parámetro. Es equivalente a escribir `{ numero -> numero * numero }`, pero más conciso.

1. **Cuándo se Utiliza**: `it` se utiliza automáticamente solo cuando la lambda tiene un solo parámetro. Si la lambda tiene más de un parámetro o si quieres nombrar explícitamente el parámetro por claridad, entonces no se puede usar `it`.
2. **Legibilidad**: Aunque `it` es útil para mantener el código conciso, en algunos casos puede ser menos legible, especialmente si el cuerpo de la lambda es largo

o complejo. En tales situaciones, puede ser mejor dar un nombre explícito al parámetro para hacer el código más fácil de entender.

Para darle un nombre a `it` en una lambda en Kotlin, simplemente defines explícitamente el parámetro de la lambda. Esto se hace colocando el nombre del parámetro seguido de una flecha (`->`) al principio de la lambda. Aquí te muestro cómo hacerlo con un ejemplo:Supongamos que tienes una lista y quieres aplicar una operación a cada elemento. Usando `it`, lo harías así:

```
val numeros = listOf(1, 2, 3)
val cuadrados = numeros.map { it * it }
```

En este caso, `it` se refiere implícitamente a cada elemento de la lista `numeros`. Pero si prefieres darle un nombre explícito al parámetro, lo haces de la siguiente manera:

```
val cuadrados = numeros.map { numero -> numero * numero }
```

Aquí, `numero` es el nombre que le has dado al parámetro que antes se referenciaba con `it`. Esto puede hacer que tu código sea más legible, especialmente si la lambda es compleja o si estás trabajando con varias lambdas y quieres evitar confusiones.

Nombrar los parámetros de las lambdas es una buena práctica cuando el contexto no es inmediatamente claro o cuando la lambda es lo suficientemente compleja como para beneficiarse de una mayor claridad.

Por tanto, `it` es una herramienta útil en Kotlin que ayuda a mantener las lambdas simples y concisas cuando solo tienen un parámetro. Es parte de lo que hace a Kotlin un lenguaje expresivo y agradable de usar.

1.5. Manejo de nulos y operador Elvis.

Controlar los nulos es una parte crucial en Kotlin, y es uno de los aspectos donde este lenguaje realmente brilla, gracias a su sistema de tipos que distingue entre referencias que pueden ser nulas (`nullable`) y las que no pueden serlo (`non-nullable`). Veamos cómo puedes manejar los nulos y usar el operador Elvis en Kotlin:

1. **Tipos Nulables y No Nulables**: En Kotlin, una variable no puede ser nula a menos que se declare explícitamente como nulable. Se agrega un signo de interrogación (`?`) al tipo para indicar que puede ser nula.
2. `var nombre: String = "Cirolele"  // No puede ser nula`
3. `var apellido: String? = null     // Puede ser nula`
4. **Chequeo de Nulos**: Antes de usar una variable nulable, debes verificar si es nula.
5. `if (apellido != null) {`
6. `    println(apellido.length)`
7. `}`
8. **Operador de Llamada Segura (`?.`)**: Permite acceder de forma segura a las propiedades y métodos de una referencia nulable. Si la referencia es nula, no se ejecuta el método o propiedad y devuelve nulo.
9. `println(apellido?.length)  // Imprime el largo si no es nulo, sino imprime null`

10. **Operador Elvis (`?:`)**: Este operador se utiliza para proporcionar un valor por defecto en caso de que la expresión a su izquierda sea nula. Es como decir "usa esto, pero si es nulo, entonces usa aquello".
11. `val longitud = apellido?.length ?: 0  // Si apellido es nulo, usa 0`

    El operador Elvis es extremadamente útil para evitar el anidamiento excesivo de declaraciones `if` y proporcionar valores predeterminados de manera concisa.

12. **Operador de Aserto No Nulo (`!!`)**: Convierte cualquier valor a un tipo no nulable, lanzando una excepción `NullPointerException` si el valor es nulo. Debe usarse con cuidado, ya que va en contra de la seguridad de nulos que Kotlin intenta proporcionar.
13. `val nombreNoNulo: String = nombre!!  // Lanza NullPointerException si nombre es nulo`
14. **let con Nulos**: El método `let` puede ser útil para trabajar con valores nulables. Si el objeto es nulo, el bloque no se ejecuta.

```
15. apellido?.let {
16.     println("El apellido tiene ${it.length} caracteres")
17. }
```

Aprender a controlar los nulos en Kotlin te ayuda a escribir código más seguro y menos propenso a errores en tiempo de ejecución. Recuerda, Kotlin está diseñado para minimizar la posibilidad de `NullPointerException`, y aprovechar estas características te permite escribir un código más robusto y limpio.

1.6. Diferencia entre Listas y Arrays

Las listas y los arrays son dos estructuras de datos comunes en Kotlin. Aunque son similares, tienen algunas diferencias importantes que es importante conocer. Veamos cuáles son: - **Listas**: Una lista es una colección ordenada de elementos. Puedes agregar o quitar elementos de una lista después de su creación. En Kotlin, las listas son inmutables por defecto, lo que significa que no puedes modificarlas después de su creación. Para crear una lista mutable, debes usar `mutableListOf` en lugar de `listOf`. Por ejemplo:

```
val lista = listOf("A", "B", "C")  // Lista inmutable
val listaMutable = mutableListOf("A", "B", "C")  // Lista mutable
```

- **Arrays**: Un array es una estructura de datos que contiene una colección de elementos del mismo tipo. Los arrays son representados por la clase `Array` y son mutables por defecto, lo que significa que puedes modificarlos después de su creación. Para crear un array inmutable, debes usar `arrayOf` en lugar de `arrayOf`. Por ejemplo:

```kotlin
val array = arrayOf("A", "B", "C")  // Array mutable
val arrayInmutable = arrayOf("A", "B", "C")  // Array inmutable
```

En Kotlin el uso de arrays es muy similar al de las listas. Puedes acceder a los elementos de un array usando el operador de indexación (`[]`). Por ejemplo:

```kotlin
val array = arrayOf("A", "B", "C")
println(array[0])  // Imprime "A"
```

También puedes modificar los elementos de un array usando el operador de indexación. Por ejemplo:

```kotlin
val array = arrayOf("A", "B", "C")
array[0] = "D"
println(array[0])  // Imprime "D"
```

Los arrays también tienen un método `size` que devuelve el número de elementos en el array. Por ejemplo:

```kotlin
val array = arrayOf("A", "B", "C")
println(array.size)  // Imprime 3
```

Los arrays también tienen un método `get` que devuelve el elemento en el índice especificado. Por ejemplo:

```kotlin
val array = arrayOf("A", "B", "C")
println(array.get(0))  // Imprime "A"
```

Los arrays también tienen un método `set` que establece el elemento en el índice especificado. Por ejemplo:

```kotlin
val array = arrayOf("A", "B", "C")
array.set(0, "D")
println(array.get(0))  // Imprime "D"
```

Los arrays también tienen un método `contains` que devuelve `true` si el array contiene el elemento especificado. Por ejemplo:

```
kotlin val array = arrayOf("A", "B", "C") println(array.contains("A"))
//
```

Por tanto, la principal diferencia entre un Array y una List en Kotlin radica en su tamaño y mutabilidad:

- **Tamaño**:
- Array: El tamaño de un Array es fijo una vez que se ha inicializado. No puedes agregar o eliminar elementos.
- List: Las listas pueden ser fijas (inmutables) o variables (mutables). Una List inmutable (listOf) no permite modificar su tamaño ni sus elementos, mientras

que una MutableList (mutableListOf) permite tanto cambiar elementos existentes como agregar o quitar elementos, modificando así su tamaño.

- **Mutabilidad**:
- Array: Puede modificar los elementos existentes (es mutable en sus elementos), pero no su tamaño.
- List: Dependiendo de si es una List o MutableList, puede ser inmutable (no permite modificaciones en sus elementos ni en tamaño) o mutable (permite todo tipo de modificaciones).
- **Uso**:
- Array: Más adecuado para tamaños fijos y operaciones de bajo nivel, donde el tamaño es conocido y no cambiará.
- List: Preferible para colecciones cuyo tamaño puede cambiar, o cuando se necesita una colección inmutable.
- **Tipos de Datos**:
- Array: Puede almacenar tipos primitivos de manera más eficiente (como IntArray, ByteArray).
- List: Trabaja con objetos, incluso para tipos primitivos (como List).

Resumiendo, si necesitas una colección de tamaño fijo o eficiencia para tipos primitivos, un Array puede ser la mejor elección. Si requieres flexibilidad en el tamaño o quieres trabajar con colecciones inmutables, entonces una List sería más adecuada.

1.7. Como crear listas

En Kotlin, hay varias formas de crear listas. Cada una tiene sus propias características y usos, y es importante conocerlas para poder elegir la más adecuada para cada situación. Veamos cómo crear listas en Kotlin:

1. **Listas Inmutables Vacías**: Para crear una lista inmutable y vacía, usas `emptyList`. Por ejemplo: `kotlin val emptyList = emptyList<String>() // Una lista vacía de Strings`
2. **Listas Inmutables**: Usas `listOf` para crear listas inmutables. Si tienes `val readOnlyList = listOf("John", "Doe")`, no puedes agregar o quitar elementos de `readOnlyList`. `listOfNotNull` se usa para excluir nulos, como en `val filteredList = listOfNotNull("A", "B", null)`, resultando en una lista con "A" y "B" solamente.
3. **Listas Mutables**: Con `mutableListOf`, creas listas a las que puedes agregar o quitar elementos después. Por ejemplo:

```
var mutableList = mutableListOf("Sydney", "Tokyo")
mutableList.add("New York") // Ahora mutableList contiene Sydney, Tokyo y New York
```

`arrayListOf` es similar, pero específicamente crea un `ArrayList`.

1. **Conversión a Listas**: Puedes convertir otras estructuras como `Map` a `List` con `toList`. Por ejemplo, al convertir un `Map` de direcciones:

```
val userAddressMap = mapOf("A" to "India", "B" to "Australia")
val addressList = userAddressMap.toList() // Convierte el Map a una List de Pares
```

1. **Constructores de Listas**: `List(size) { lambda }` crea una lista inmutable con un tamaño definido y elementos inicializados por la lambda. `MutableList` es similar pero mutable. Por ejemplo:

```
val myList = List(3) { it * 2 } // Crea una lista [0, 2, 4]
```

`buildList` permite construir una lista inmutable con un bloque mutable. Ejemplo:

```
val builtList = buildList {
    add("One")
    add("Two")
} // Crea una lista ["One", "Two"]
```

Cada uno de estos métodos tiene usos específicos, dependiendo de si necesitas modificar la lista después de su creación o si necesitas inicializarla de una manera particular.

1.8 Listas de listas

Crear matrices o listas bidimensionales en Kotlin es bastante sencillo. Una lista bidimensional es básicamente una lista de listas. Aquí te muestro cómo puedes hacerlo:

1. **Creación de una Lista Bidimensional Manualmente**: Puedes crear una lista bidimensional manualmente definiendo una lista de listas. Por ejemplo, para crear una matriz 2x3:

```
val matriz = listOf(
    listOf(1, 2, 3),
    listOf(4, 5, 6)
)
// Aquí matriz es una lista de dos listas, cada una con tres elementos.
```

1. **Creación Dinámica**: Si quieres crear una matriz con tamaños dinámicos, puedes usar bucles para inicializarla. Por ejemplo, crear una matriz de 3x3:

```
val filas = 3
val columnas = 3
val matrizDinamica = List(filas) { MutableList(columnas) { 0 } }
// Crea una matriz 3x3 con todos los elementos inicializados a 0.
```

1. **Acceso y Modificación de Elementos**: Puedes acceder o modificar los elementos de la matriz usando índices. Por ejemplo, para cambiar un elemento:

```
matrizDinamica[0][1] = 10  // Cambia el segundo elemento de la primera fila a 10
```

Y para leer un elemento:

```
val elemento = matrizDinamica[0][1]  // Lee el segundo elemento de la primera fila
```

1. **Iteración sobre Elementos**: Puedes iterar sobre los elementos de la matriz utilizando bucles anidados:

```
for (fila in matrizDinamica) {
    for (elemento in fila) {
        println(elemento)
    }
}
```

Estas son las formas básicas de trabajar con matrices o listas bidimensionales en Kotlin. Puedes adaptar estos ejemplos según tus necesidades específicas, como cambiar el tipo de datos almacenados o modificar la forma de inicialización.